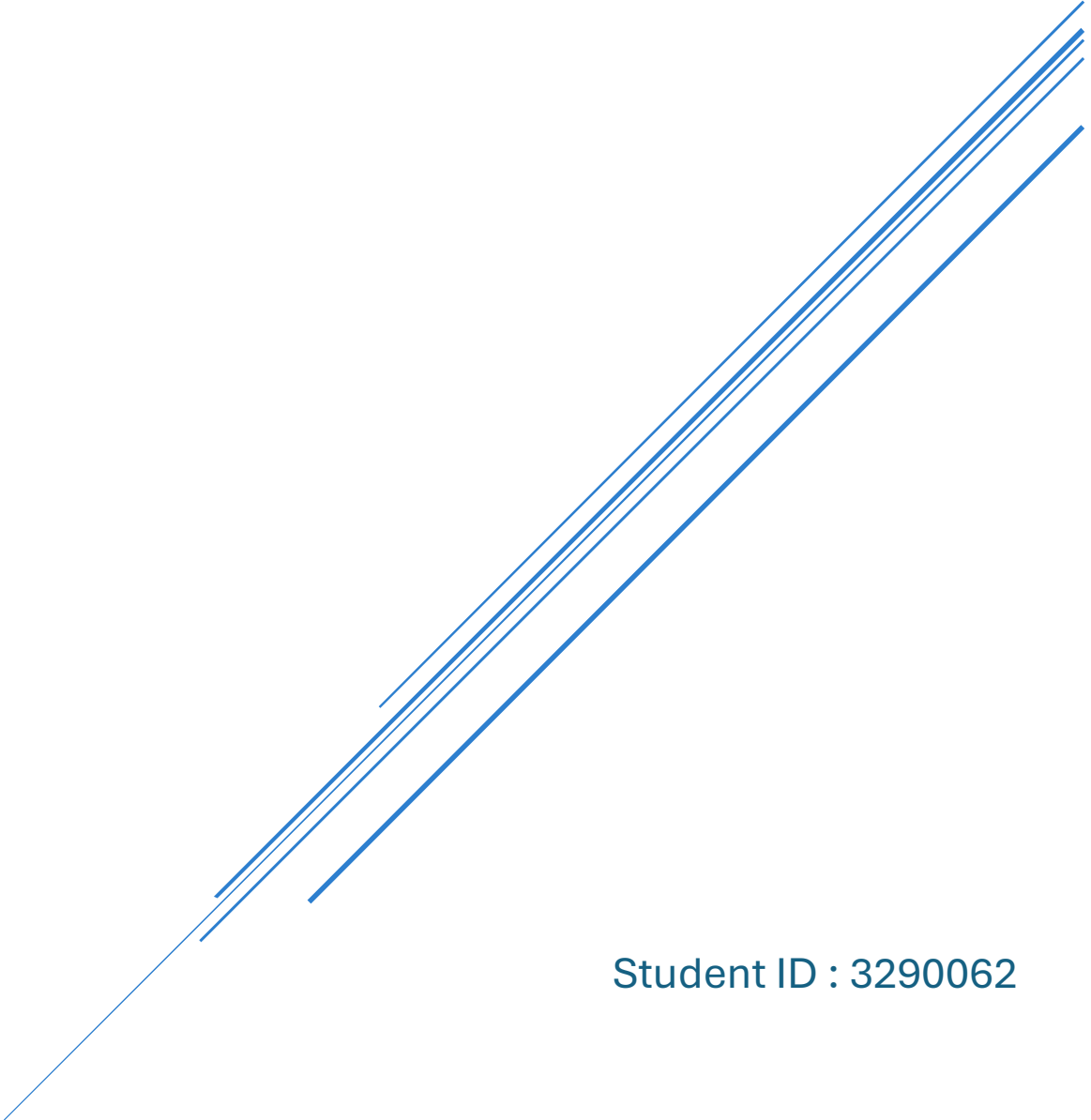


DSA EXPLORER AND VISUALISER APP

Software Technology 2 (7170)



Student ID : 3290062

Introduction

Project Overview

The DSA Explorer & Visualizer App is an engaging educational program that aims to create an interface between DS&A in theory and their implementation in practice. Using Python and the pygame module, this application allows for interactive work with basic data structures, viewing of the execution process of algorithms in action, and solving pathfinding problems.

Application Architecture and Modularity

In order to achieve the highest standards of code quality and its maintainability, this application was developed on the basis of strict modularity and Object-Oriented Programming (OOP). The application structure is based on a clear principle of the Separation of Concerns (SoC), which involves the following application layers:

- **Core Logic Layer (`core_*.py`):** pure Python code for the data structures and algorithms' implementations without any pygame code inside them, which makes them easily testable;
- **UI/Presentation Layer (`phase*_ui.py`):** Pygame classes, which take care of rendering operations, coordinate calculations, and interactions.

These layers communicate via the central `main.py` file, which acts as a finite-state machine of navigating through the three main application modules.

Phase 1: Data Structures Playground

Implementation and Logic

Phase 1 starts with basic data structures like Stacks, Queues, Singly Linked Lists, and Binary Search Trees (BST). The underlying logic for these structures is implemented inside the `core_dsa.py` script using object-oriented programming techniques. A rigid separation of concerns guarantees that the mathematical properties of the data (such as connections between nodes, boundary conditions, etc.) are handled entirely separately from the Pygame loop.

Interactive Visualization (`phase1_ui.py`)

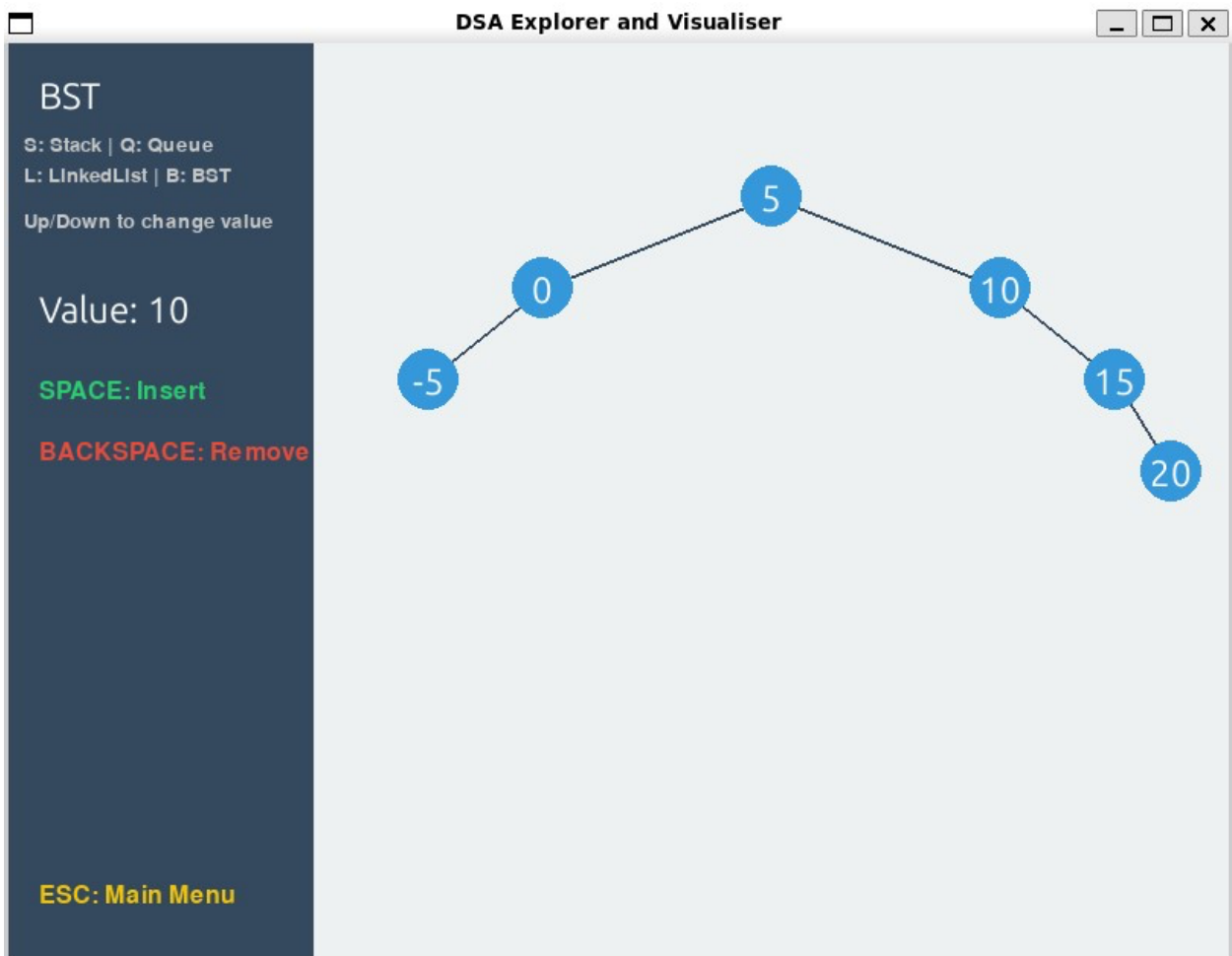
The UI component offers a live, interactive visualization for each data structure:

- **Stacks & Queues:** Visualization occurs in live containers that react instantaneously to any insertions or deletions performed by pressing the assigned key combination.
- **Singly Linked List:** Offers arrow-based visualization to demonstrate node relationships. There is also an interesting interactive element, allowing a user to point their cursor to the desired location within the list and then perform an exact `insert_at_pos` action.
- **BST:** Makes use of a recursive algorithm for automatic balancing. During tree manipulation, this algorithm calculates `(x, y)` offsets that ensure no overlapping at any level.

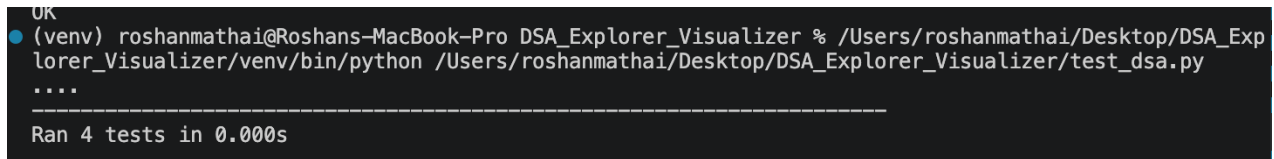
Automated Testing

Unit tests were developed in Python using the `unittest` library as part of the `test_dsa.py` script for Phase 1's data structures. Tests run successfully and verify the necessary properties without relying on the visualization interface:

- Checking Stack LIFO order in push/pop sequences.
- Checking Queue FIFO order in enqueue/dequeue sequences.
- Checking positional insertion of elements in linked lists.
- Validating BST insertions and in-order traversal results.



Caption: Figure 1: Pygame visualization of the Phase 1 Data Structure Playground

A terminal window with a dark background. The prompt is a blue circle followed by the user 'roshanmathai' and the machine 'Roshans-MacBook-Pro'. The command is 'DSA_Explorer_Visualizer % /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/venv/bin/python /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/test_dsa.py'. The output shows '....' followed by a dashed line and 'Ran 4 tests in 0.000s'.

```
OK
● (venv) roshanmathai@Roshans-MacBook-Pro DSA_Explorer_Visualizer % /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/venv/bin/python /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/test_dsa.py
....
-----
Ran 4 tests in 0.000s
```

Caption: Figure 2: Automated unit test results for Phase 1 data structures.

Phase 2: Algorithm Visualizer

Sorting Algorithms and Generator Animation

Phase 2 implements Bubble Sort, Selection Sort, and Merge Sort in a recursive fashion.

One of the hardest challenges for animation within algorithms in Pygame is not letting the while/for loop of the algorithm block the event handler loop of the application (otherwise the window freezes).

For this, as well as the achievement of smooth animations, we used Python Generators (`yield`). Instead of sorting the whole array once in place, the generator sorts part of it by returning the array's state (together with indices of the compared elements). Pygame then gets this returned array state and renders the array in real time, highlighting the compared elements with red color.

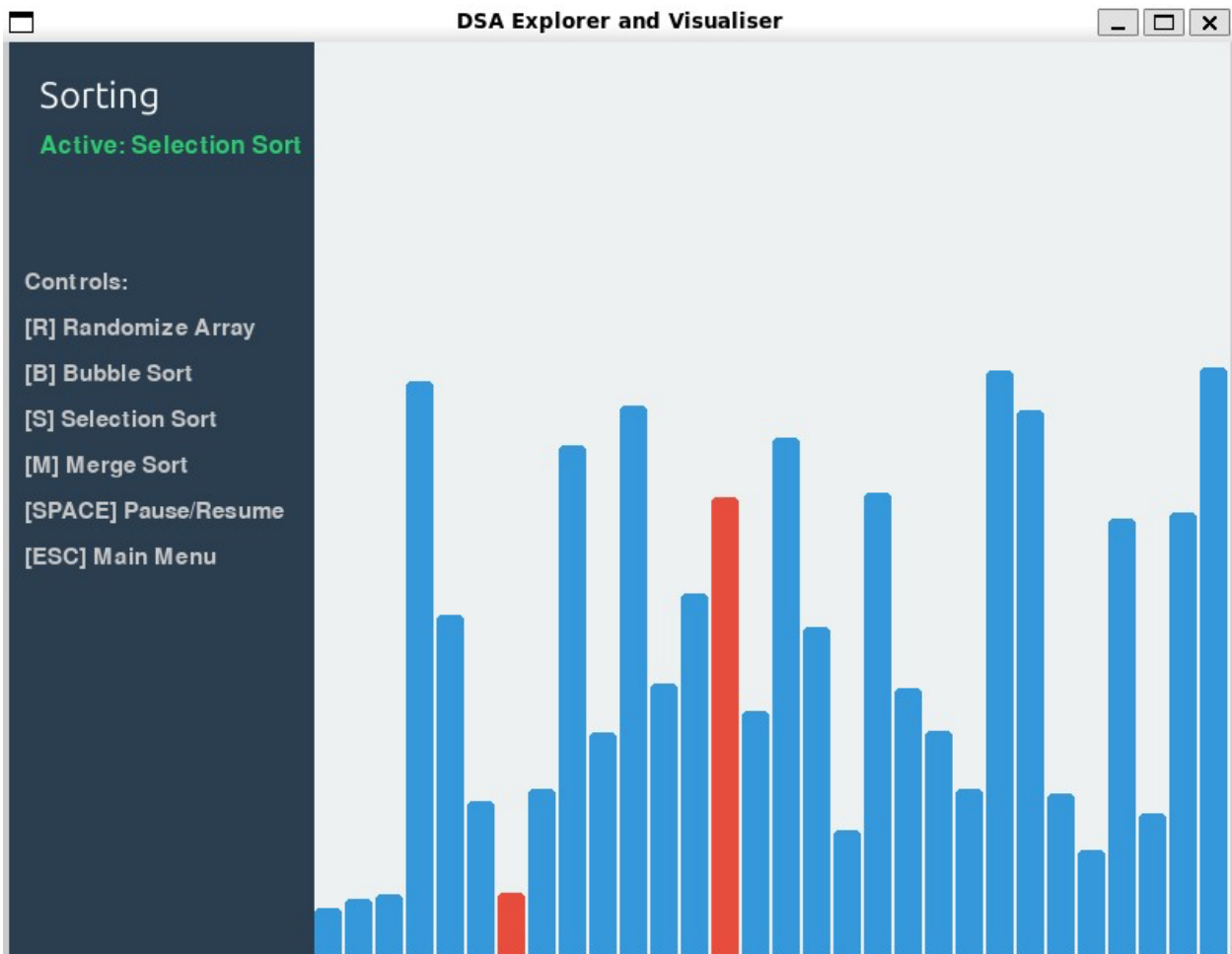
Graph Traversals

Graph module traverses the graph either by using BFS or DFS, depending on the user's choice (undirected graph). The UI allows users to choose any particular node and select it

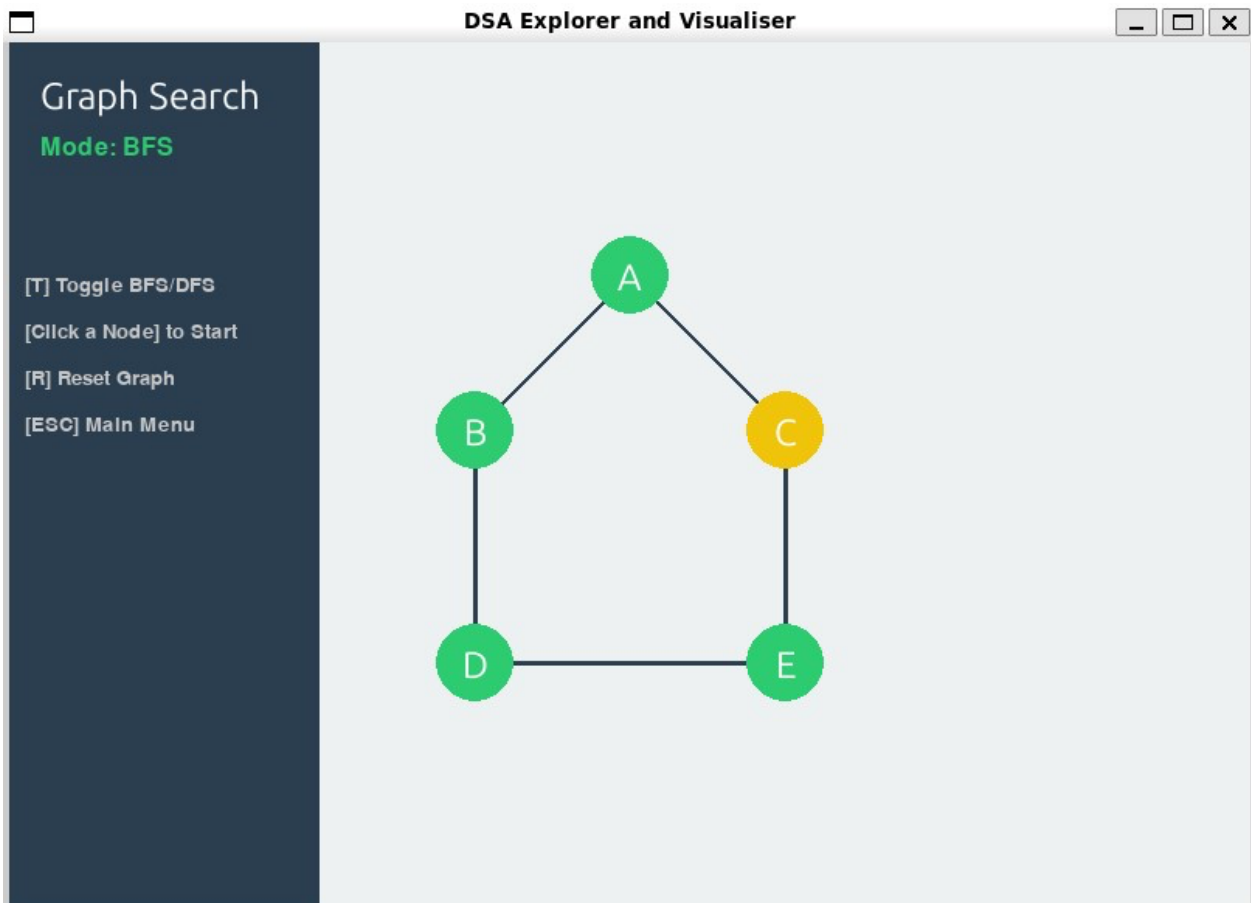
as their starting node. It becomes clear after visualizing both types of search how the BFS and DFS work differently: one expands level by level and the other dives deeper into the branches.

Heap Visualizer

Max Heap is implemented using an ordinary array as its backend. Pygame UI transforms this linear array into a binary max heap (complete binary tree), using logarithmic coordinate calculation (`math.log2`). On insertion or extraction of a maximal item, the Pygame UI visualizes how the process of bubbling up/down works.



Caption: Figure 3: Selection Sort execution, highlighting the active elements being compared.



Caption: Figure 4: BFS Traversal originating from a user-selected start node

Phase 3: Puzzle Challenges

Integration of Multiple DS&A Concepts for a Complex Problem Solution

The last step is about applying all DS&A concepts together to tackle a challenging problem.

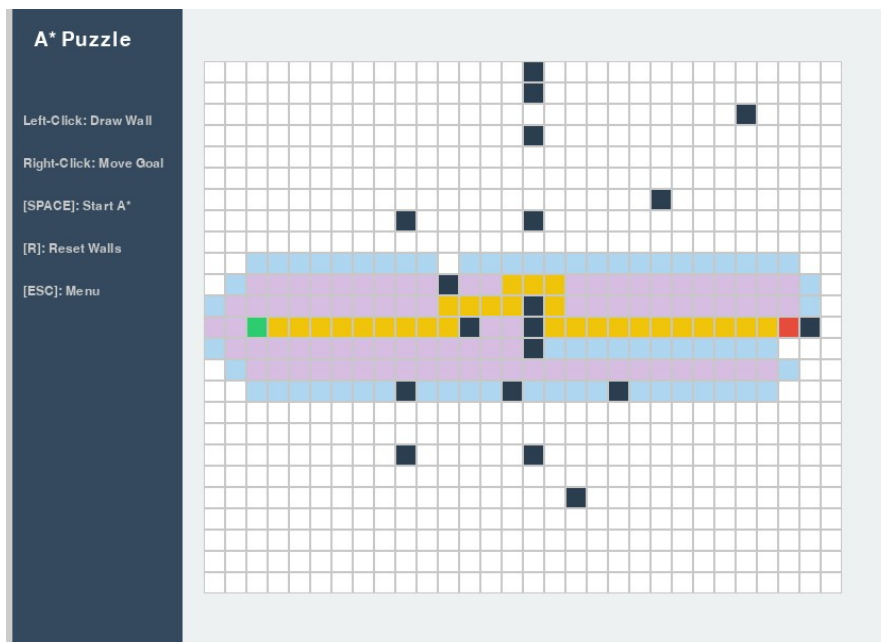
In this case, the selected puzzle is a grid-based interactive path-finding puzzle using the *A Search Algorithm**.

It involves:

1. **Graphs:** The 2D grid serves as an implicit graph where each square is linked to its orthogonally neighboring squares.
2. **Heaps:** The Min-Heap implemented by Python's native `heapq` is utilized as the Priority Queue that always evaluates the best possible path according to the lowest f-score.
3. **Heuristics:** Manhattan Distance Heuristic is employed to help find the optimal route to the destination.

User Interaction with DS&A Module

The UI enables users to create their own walls by simply drawing lines using the mouse, as well as move the goal node through right-click. Once executed, the A* Algorithm will then trace a route from start to finish, avoiding any obstacles that may be created by the users, distinguishing between the Open Set, Closed Set, and the shortest path found.



(Caption: Figure 5: A Pathfinding algorithm dynamically avoiding user-drawn obstacles to find the shortest path.)*

Automated Testing and Benchmarking

Unit Testing Framework

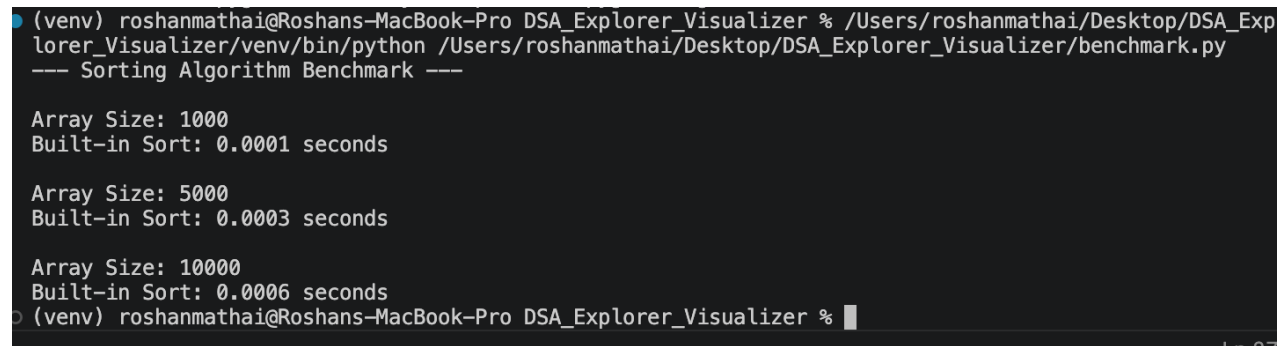
For testing purposes, a suite of unit tests was written to automatically assess the correctness and reliability of the underlying logic, leveraging the standard library module `unittest`. Given the strict adherence to the modular structure, it became possible to isolate and import for testing the pure logic modules (`core_dsa.py`, `core_graph.py`, `core_puzzles.py`) without involving the Pygame rendering environment.

Tests were created for all the three phases of the task as follows:

- **Phase 1:** Assesses whether LIFO/FIFO restrictions were fulfilled, whether the pointers were manipulated properly when doing the positional linked list insertions and whether BST nodes are returned in the in-order traversal.
- **Phase 2:** Tests whether BFS and DFS traverse graphs properly returning nodes in an expected order, and that the maximum heap returns elements sorted in a descending order.
- **Phase 3:** Tests whether the A* pathfinding algorithm avoids coordinate based obstacles in its search and does not cross them.

Benchmarking

Along with unit testing, a benchmarking script (`benchmark.py`) was created to analyze the time complexities of the implemented algorithms in practice. It creates arrays of various sizes (e.g., 1000, 5000, 10000 elements) and compares the time taken by the sort implemented and the native one in Python. This gave an understanding of the time complexities of the Selection and Bubble sorts being $O(n^2)$ while merge sort is $(n \log n)$

A terminal window showing the execution of a benchmarking script. The prompt is '(venv) roshanmathai@Roshans-MacBook-Pro DSA_Explorer_Visualizer %'. The command executed is '/Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/venv/bin/python /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/benchmark.py'. The output shows three test cases for array sizes 1000, 5000, and 10000, each comparing a 'Built-in Sort' with a custom implementation. The times are very low, indicating efficient implementations.

```
(venv) roshanmathai@Roshans-MacBook-Pro DSA_Explorer_Visualizer % /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/venv/bin/python /Users/roshanmathai/Desktop/DSA_Explorer_Visualizer/benchmark.py
--- Sorting Algorithm Benchmark ---

Array Size: 1000
Built-in Sort: 0.0001 seconds

Array Size: 5000
Built-in Sort: 0.0003 seconds

Array Size: 10000
Built-in Sort: 0.0006 seconds
(venv) roshanmathai@Roshans-MacBook-Pro DSA_Explorer_Visualizer %
```

(Caption: Figure 6: Terminal output of the algorithm benchmarking script.)

Conclusion

The implementation of DSA Explorer and Visualizer successfully linked theoretical algorithm design with actual software engineering practice. This was made possible through heavy use of Object-Oriented programming, state machines, and Python generators. As a result, the app delivered smooth and non-blocking animations of data manipulations visually.

The strong distinction between the math behind and the frontend using Pygame led to the creation of very testable codebase. Finally, combining everything in a puzzle called A* Pathfinding puzzle ensured comprehension of how different data structures (Priority Queue and Graph) work together to solve a problem.